SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

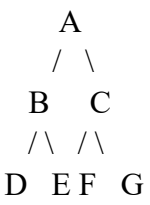
*I **Ramchandran V(192321108)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ramchandran V

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



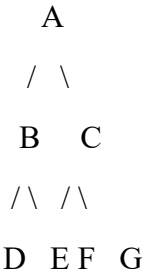
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

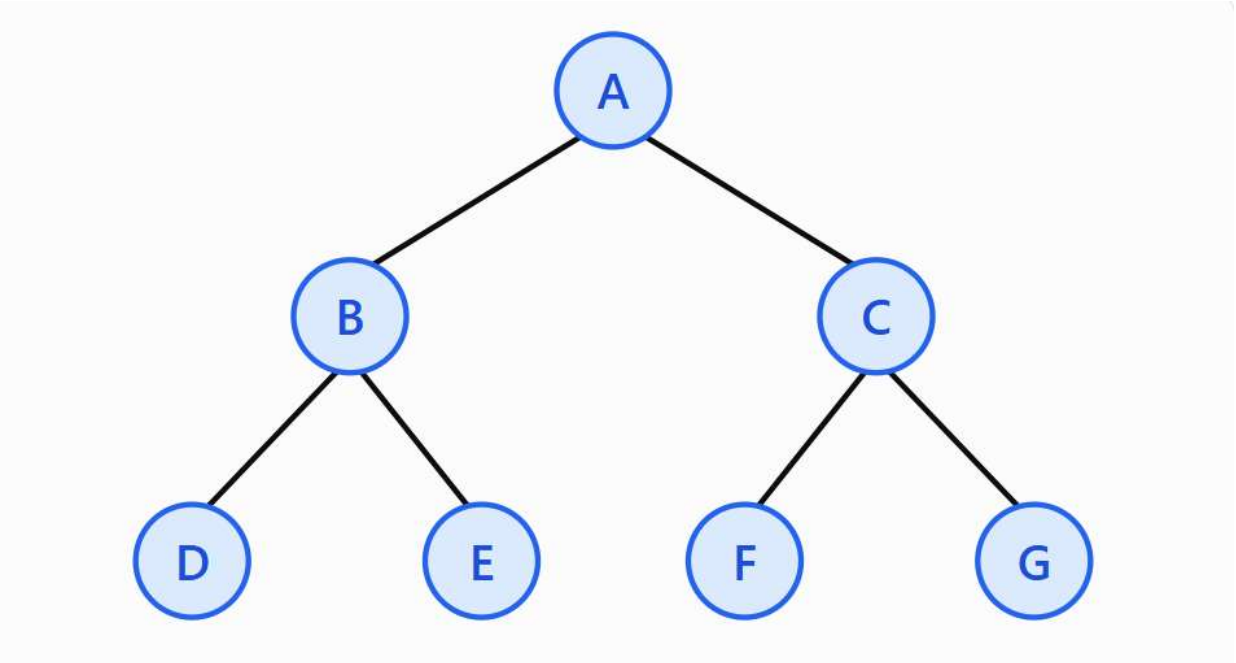
Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
Criteria	**Weightage (%)**	**Level 4 – Exemplary (4)**	**Level 3 – Proficient (3)**	**Level 2 – Developing (2)**	**Level 1 – Beginning (1)**
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1.Breadth-First Search (BFS)

Aim

To perform Breadth-First Search on the given graph, starting from node A, and record the traversal order and queue contents after each iteration.

Given graph



The graph contains seven nodes: A, B, C, D, E, F, and G.

- A is the root node.
- B and C are the children of A.
- D and E are the children of B.
- F and G are the children of C.

Algorithm

BFS visits nodes level by level, from left to right. It uses a queue that follows the FIFO (First In, First Out) principle.

1. Insert the starting node A into the queue.
2. Remove the node at the front of the queue.
3. Mark that node as visited.
4. Insert its unvisited children at the rear of the queue, from left to right.
5. Repeat until the queue is empty.

Detailed dry run

Iteration	Node removed and visited	Queue after expansion	Visited nodes
1	A	B, C	A
2	B	C, D, E	A, B

3	C	D, E, F, G	A, B, C
4	D	E, F, G	A, B, C, D
5	E	F, G	A, B, C, D, E
6	F	G	A, B, C, D, E, F
7	G	Empty	A, B, C, D, E, F, G

Step-by-step explanation

Iteration 1: The queue initially contains A. Remove A and visit it. Add its children B and C. The queue becomes [B, C].

Iteration 2: Remove B and visit it. Add its children D and E behind C. The queue becomes [C, D, E].

Iteration 3: Remove C and visit it. Add its children F and G. The queue becomes [D, E, F, G].

Iteration 4: Remove D and visit it. D is a leaf node, so no new nodes are added. The queue becomes [E, F, G].

Iteration 5: Remove E and visit it. Since E has no children, the queue becomes [F, G].

Iteration 6: Remove F and visit it. The queue becomes [G].

Iteration 7: Remove G and visit it. The queue becomes empty, so the algorithm terminates.

5. Answers to the questions

1. Write the BFS traversal order.

A → B → C → D → E → F → G

2. Show the queue contents after each iteration.

- After iteration 1: [B, C]
- After iteration 2: [C, D, E]
- After iteration 3: [D, E, F, G]
- After iteration 4: [E, F, G]
- After iteration 5: [F, G]
- After iteration 6: [G]
- After iteration 7: []

Result: BFS visits all seven nodes in level order, starting from A.

2. Depth-First Search (DFS)

Aim

To perform Depth-First Search on the same graph, starting from node A, and record the order in which nodes are visited.

Principle

DFS explores one branch as deeply as possible before returning to explore another branch. It uses a stack (Last In, First Out) or recursion.

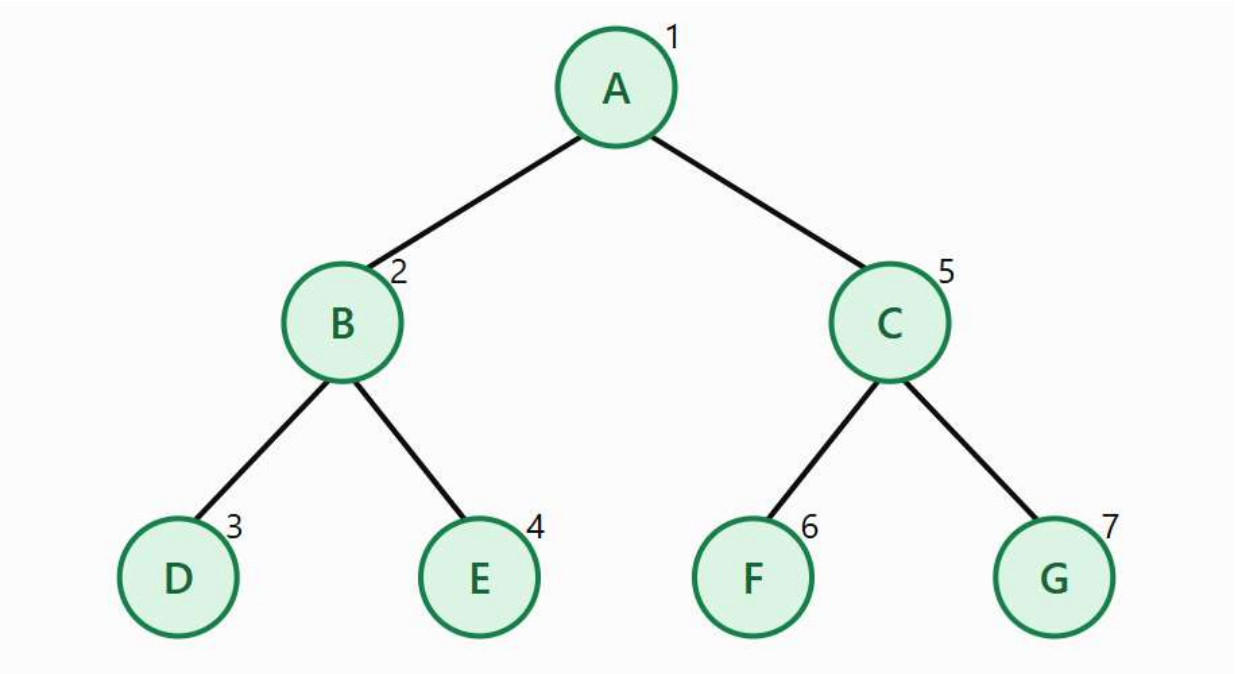
For this assessment, we visit the left child before the right child.

Algorithm

1. Start at node A.
2. Mark A as visited.
3. Move to the leftmost unvisited child.

4. Continue deeper until reaching a node with no unvisited children.
5. Backtrack to the nearest node with an unvisited child.
6. Continue until all nodes have been visited.

DFS traversal diagram



Detailed dry-run table

Iteration	Node visited	Action performed	Visited order
1	A	Move to left child B	A
2	B	Move to left child D	A, B
3	D	Leaf node; backtrack to B	A, B, D
4	E	Visit right child of B; backtrack	A, B, D, E
5	C	Backtrack to A and enter right subtree	A, B, D, E, C
6	F	Visit left child of C	A, B, D, E, C, F
7	G	Visit right child of C	A, B, D, E, C, F, G

Step-by-step explanation

Iteration 1: Start at A and mark it as visited. Move to B.

Iteration 2: Visit B. Move deeper into its left child D.

Iteration 3: Visit D. It has no children, so return to B.

Iteration 4: Visit E, the right child of B. E is also a leaf, so return to A.

Iteration 5: Move to C, the right child of A.

Iteration 6: Visit F, the left child of C. Since F has no children, backtrack to C.

Iteration 7: Visit G, the right child of C. All nodes have now been visited.

Answers to the questions

1. Write the DFS traversal order.

A → B → D → E → C → F → G

2. Explain how DFS works.

DFS goes as deep as possible along one branch. When it reaches a leaf or a node without unvisited children, it backtracks and explores the next available branch.
Result: DFS visits all seven nodes by completely exploring the left subtree before the right subtree.

3. 8-Puzzle Problem Using BFS

Aim

To solve the 8-puzzle problem using Breadth-First Search, starting from the given initial state and reaching the goal state. The task involves tracing the queue, identifying expanded states, and finding the shortest solution path.

Initial state and goal state

The 8-puzzle consists of eight numbered tiles and one blank space represented by 0. A legal move slides a tile into the blank space.

Initial state		
1	3	6
5	_	2
4	7	8

Goal state		
1	2	3
4	5	6
7	8	_

Initial state:

$$\begin{bmatrix} 1 & 3 & 6 \\ 5 & 0 & 2 \\ 4 & 7 & 8 \end{bmatrix}$$

Goal state:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

BFS algorithm for the 8-puzzle

1. Insert the initial state into a FIFO queue.
2. Mark the initial state as discovered.
3. Remove the state at the front of the queue.
4. Check whether the current state matches the goal.
5. If it does not, generate all legal blank moves.

6. Add new states that have not been discovered to the rear of the queue.
7. Store each state's parent and move so that the solution path can be reconstructed.
8. Continue until the goal state is reached.

For the dry run below, successors are generated in this order: Up → Down → Left → Right. Illegal moves and previously discovered states are skipped.

Detailed solution path

	1	3	6
S0	5	_	2
Start	4	7	8
Initial state			
	1	3	6
S1	5	2	_
Step 1	4	7	8
Right			
	1	3	_
S2	5	2	6
Step 2	4	7	8
Up			
	1	_	3
S3	5	2	6
Step 3	4	7	8
Left			
	1	2	3
S4	5	_	6
Step 4	4	7	8
Down			
	1	2	3
S5	_	5	6
Step 5	4	7	8
Left			
	1	2	3
S6	4	5	6
Step 6	_	7	8
Down			
	1	2	3
S7	4	5	6
Step 7	7	_	8
Right			
	1	2	3
S8	4	5	6
Goal	7	8	_

Right – Goal reached

The blank space moves in the following order:

Right → Up → Left → Down → Left → Down → Right → Right

The path has nine states, including the initial and goal states, and requires eight moves.

BFS queue dry run: first five iterations

The assessment asks for the current state, queue before expansion, and queue after expansion. To keep the table readable, the states below use short labels.

Label	Board state
S0	136 / 502 / 478
U1	106 / 532 / 478
D1	136 / 572 / 408
L1	136 / 052 / 478
R1	136 / 520 / 478

In these labels, each three-digit group represents one row. For example, 136 / 502 / 478 means the rows are 1 3 6, 5 0 2, and 4 7 8.

Dry-run table

Iteration	Current state expanded	Queue before expansion	Queue after expansion
1	S0	[S0]	[U1, D1, L1, R1]
2	U1	[U1, D1, L1, R1]	[D1, L1, R1, U2, UR2]
3	D1	[D1, L1, R1, U2, UR2]	[L1, R1, U2, UR2, DL2, DR2]
4	L1	[L1, R1, U2, UR2, DL2, DR2]	[R1, U2, UR2, DL2, DR2, LU2, LD2]
5	R1	[R1, U2, UR2, DL2, DR2, LU2, LD2]	[U2, UR2, DL2, DR2, LU2, LD2, RU2, RD2]

The additional labels denote newly generated states:

- U2, UR2: states generated from U1.
- DL2, DR2: states generated from D1.
- LU2, LD2: states generated from L1.
- RU2, RD2: states generated from R1.

Only newly discovered states are added to the queue. A state already discovered is not inserted a second time.

Important: The five rows above show the first five expansions, not the complete search. BFS continues expanding states in FIFO order until the goal state is dequeued.

Answers to the questions

1. Which node is expanded in each iteration?

During the first five iterations, the expanded states are:

1. S0 – the initial state.
2. U1 – the state generated by moving the blank up.
3. D1 – the state generated by moving the blank down.
4. L1 – the state generated by moving the blank left.
5. R1 – the state generated by moving the blank right.

After these iterations, BFS continues with the remaining states in queue order until it reaches the goal.

2. How many states are generated in total?

Using the stated successor order, counting the initial state and counting each unique discovered board only once, the search discovers 500 unique states by the time the goal is dequeued. Of these, 311 states are expanded, including the goal state.

The number can differ if an assessment uses a different definition of "generated state," such as counting repeated successor attempts or excluding the initial state.

3. Write the complete solution path from the initial state to the goal state.

The shortest path is:

S0 → S1 → S2 → S3 → S4 → S5 → S6 → S7 → S8

The blank-space moves are:

Right → Up → Left → Down → Left → Down → Right → Right

The goal state is:

The goal state is:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores states in increasing order of depth. It first explores the initial state, then all states one move away, then all states two moves away, and so on.

Since every legal move has equal cost, the first goal state reached has the minimum number of moves. Therefore, BFS guarantees a shortest solution when the state space is finite and duplicate states are handled correctly.

Final conclusion

Breadth-First Search explores nodes level by level using a queue, while Depth-First Search explores deeply along one branch before backtracking. In the 8-puzzle problem, BFS systematically explores states and identifies a shortest solution of eight moves for the given initial and goal states.

Both BFS and DFS are fundamental AI search algorithms. BFS is suitable when the shortest path is required in an unweighted problem, whereas DFS is useful for exploring search trees with lower memory requirements in many cases.

Annexure B – Python Implementation and Output Screenshots

Name: Ramchandran V Reg No: 192321108 Course: Artificial Intelligence (CSA17) CO2
Each program below was executed and the results were verified against the manual dry runs given earlier in this document (BFS order, DFS order, 8-move solution path, 500 states discovered, 311 states expanded).

Question 1 – Breadth-First Search (BFS) – Python Program and Output

The program uses a FIFO queue (`collections.deque`). A node is removed from the front, marked visited, and its unvisited children are added at the rear from left to right. The printed table matches the dry run in Question 1.

Python code

q1_bfs.py

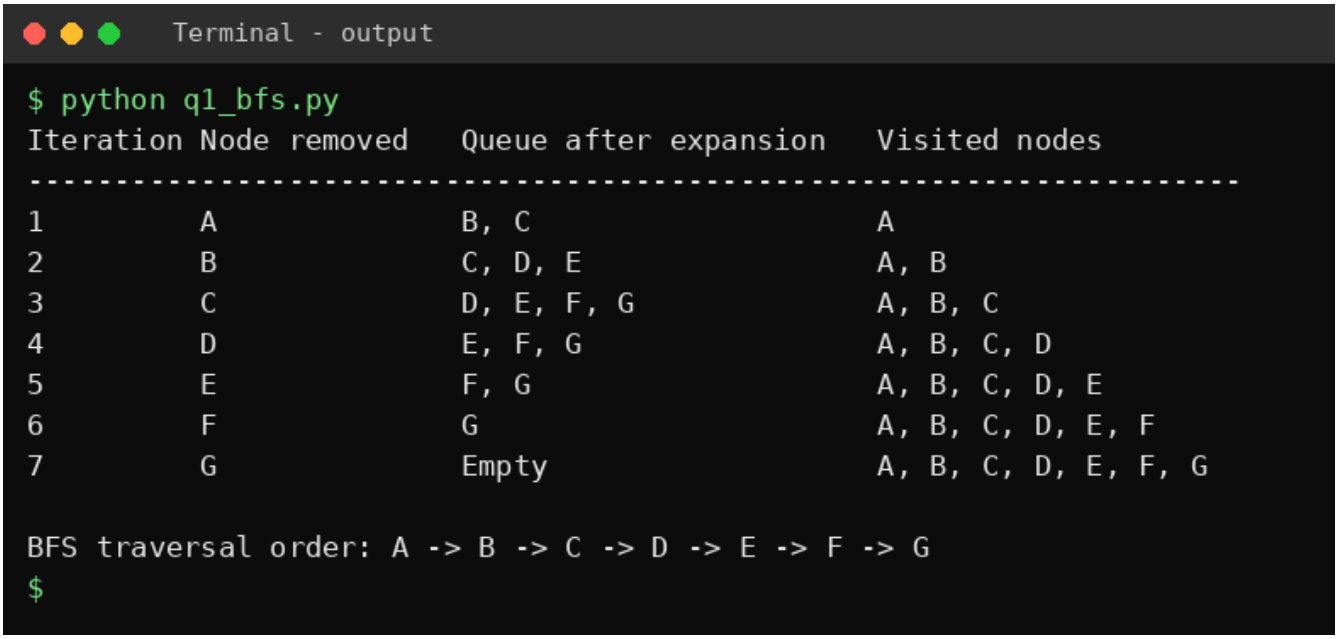
```python
# Question 1 - Breadth-First Search (BFS) on the given tree
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [], "E": [], "F": [], "G": [],
}

def bfs(graph, start):
    queue = deque([start])          # FIFO queue
    visited = []
    print(f"{'Iteration':<10}{'Node removed':<15}{'Queue after expansion':<24}Visited nodes")
    print("-" * 70)
    it = 0
    while queue:
        it += 1
        node = queue.popleft()      # remove from front
        visited.append(node)        # mark visited
        for child in graph[node]:   # add children at rear, left to right
            if child not in visited and child not in queue:
                queue.append(child)
        q = ", ".join(queue) if queue else "Empty"
        print(f"{it:<10}{node:<15}{q:<24}{', '.join(visited)}")
    return visited

order = bfs(graph, "A")
print("\nBFS traversal order:", " -> ".join(order))
```

Question 1 – Breadth-First Search (BFS) – Output Screenshot



Result: BFS traversal order = A → B → C → D → E → F → G. The queue after each iteration is [B, C], [C, D, E], [D, E, F, G], [E, F, G], [F, G], [G], [ ], exactly as in the manual dry run.

Question 2 – Depth-First Search (DFS) – Python Program and Output

The program uses a LIFO stack. Children are pushed in reverse order so the left child is popped first. The stack column (bottom to top) fills the “Stack” column of the Question 2 table; a recursive version confirms the same order.

Python code

q2_dfs.py

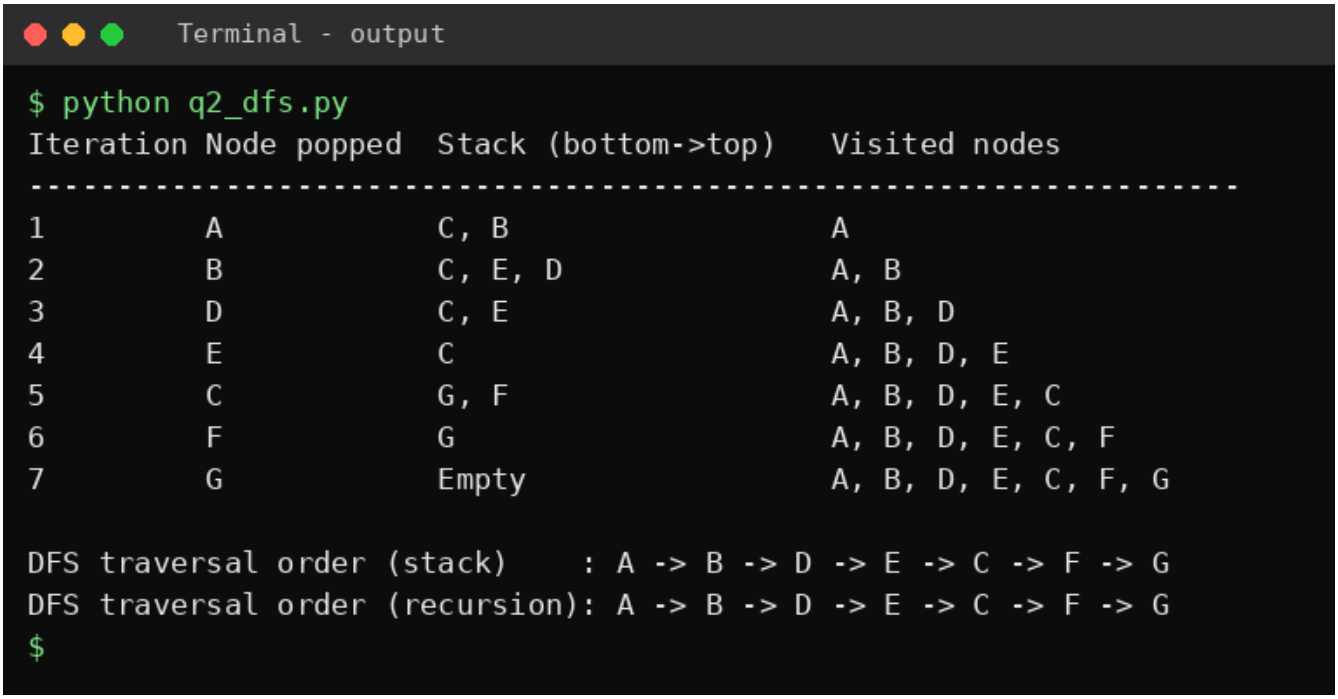
```python
# Question 2 - Depth-First Search (DFS) on the same tree (left child first)
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [], "E": [], "F": [], "G": [],
}

def dfs(graph, start):
    stack = [start]                # LIFO stack (top = right end)
    visited = []
    print(f"{'Iteration':<10}{'Node popped':<13}{'Stack (bottom->top)':<22}Visited nodes")
    print("-" * 68)
    it = 0
    while stack:
        it += 1
        node = stack.pop()         # remove from top
        visited.append(node)
        # push children in reverse so the LEFT child is popped first
        for child in reversed(graph[node]):
            if child not in visited and child not in stack:
                stack.append(child)
        s = ", ".join(stack) if stack else "Empty"
        print(f"{it:<10}{node:<13}{s:<22}{', '.join(visited)}")
    return visited

def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = []
    visited.append(node)
    for child in graph[node]:
        if child not in visited:
            dfs_recursive(graph, child, visited)
    return visited

order = dfs(graph, "A")
print("\nDFS traversal order (stack)    :", " -> ".join(order))
print("DFS traversal order (recursion):", " -> ".join(dfs_recursive(graph, "A")))
```

Question 2 – Depth-First Search (DFS) – Output Screenshot



Result: DFS traversal order = A → B → D → E → C → F → G. The stack (top at the right) is [C, B], [C, E, D], [C, E], [C], [G, F], [G], [ ]; the recursive version gives the same order.

Question 3 – 8-Puzzle using BFS – Python Program and Output

The program runs BFS from the initial state 136/502/478 to the goal 123/456/780 with successor order Up → Down → Left → Right, skips already-discovered states, and rebuilds the path from parent links. Each state is written row-wise (136/502/478 = rows 1 3 6, 5 0 2, 4 7 8).

Python code

q3_8puzzle.py (part 1)

```python
# Question 3 - 8-Puzzle solved with Breadth-First Search
from collections import deque

START = (1, 3, 6, 5, 0, 2, 4, 7, 8)
GOAL  = (1, 2, 3, 4, 5, 6, 7, 8, 0)
MOVES = [("Up", -3), ("Down", 3), ("Left", -1), ("Right", 1)]  # successor order

def show(s):                                    # 136/502/478 style
    return "/".join("".join(map(str, s[i:i + 3])) for i in (0, 3, 6))

def successors(state):                          # legal moves of the blank (0)
    b = state.index(0)
    r, c = divmod(b, 3)
    for name, d in MOVES:
        if (name == "Up" and r == 0) or (name == "Down" and r == 2): continue
        if (name == "Left" and c == 0) or (name == "Right" and c == 2): continue
        s = list(state)
        s[b], s[b + d] = s[b + d], s[b]
        yield name, tuple(s)

def bfs(start, goal, trace_iterations=5):
    queue = deque([start])
    parent = {start: (None, None)}              # state -> (parent, move)
    expanded = 0
    print(f"{'It':<4}{'Current state':<15}{'Queue before':<14}Queue after   (new states added below)")
    while queue:
        before = len(queue)
        cur = queue.popleft()
        expanded += 1
```

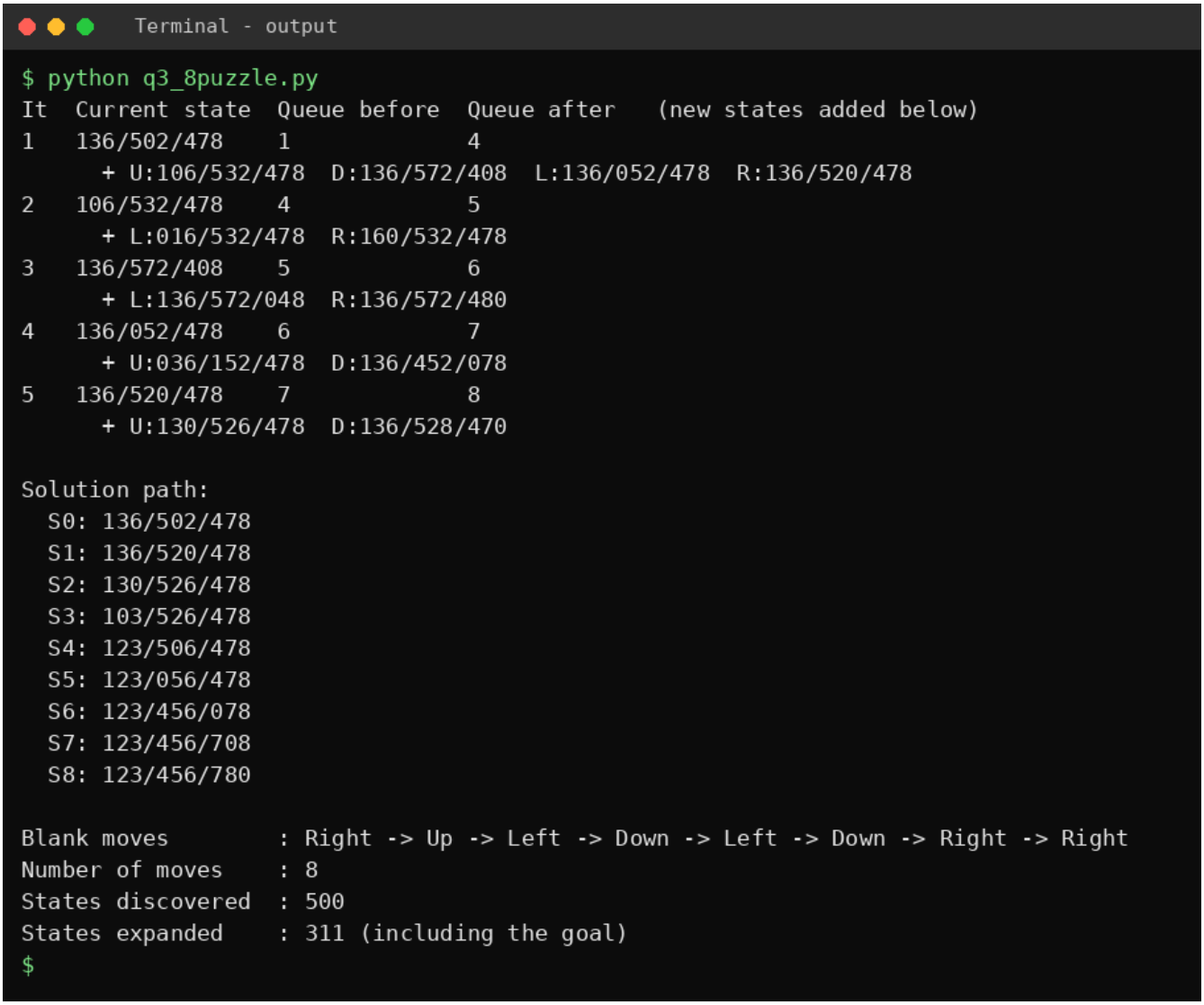
q3_8puzzle.py (part 2)

```
        if cur == goal:
            break
        new = []
        for move, nxt in successors(cur):
            if nxt not in parent:              # skip already discovered states
                parent[nxt] = (cur, move)
                queue.append(nxt)
                new.append(f"{move[0]}:{show(nxt)}")
        if expanded <= trace_iterations:
            print(f"{expanded:<4}{show(cur):<15}{before:<14}{len(queue)}")
            print("      + " + "  ".join(new))
    # rebuild solution path
    path, moves = [], []
    while cur is not None:
        path.append(cur)
        cur, m = parent[cur]
        if m: moves.append(m)
    return path[::-1], moves[::-1], len(parent), expanded

path, moves, generated, expanded = bfs(START, GOAL)

print("\nSolution path:")
for i, s in enumerate(path):
    print(f"  S{i}: {show(s)}")
print("\nBlank moves        :", " -> ".join(moves))
print("Number of moves    :", len(moves))
print("States discovered  :", generated)
print("States expanded    :", expanded, "(including the goal)")
```

Question 3 – 8-Puzzle using BFS – Output Screenshot



Result: The first five expanded states are S0, U1, D1, L1 and R1 (queue sizes 1→4, 4→5, 5→6, 6→7, 7→8). The shortest solution has 8 moves (Right → Up → Left → Down → Left → Down → Right → Right), with 500 states discovered and 311 expanded including the goal. In the table, U/D/L/R mark the blank-tile move that produced each new state.